

Multi-Agent Yield Analysis For Circuit Design

Haiyan Qin*, Jing Kou*, Liang Zhang*, Wang Kang*[§], Wei W. Xing^{†§}

^{*}National Key Laboratory of Spintronics, Hangzhou International Innovation Institute;
School of Integrated Circuit Science and Engineering, Beihang University, China

[†]School of Mathematical and Physical Sciences, University of Sheffield, Sheffield, United Kingdom

^{*}{haiyanq, koujing, zhangliang24, wang.kang}@buaa.edu.cn, [†]w.xing@sheffield.ac.uk

Abstract—Semiconductor yield estimation presents a critical challenge in modern manufacturing, directly impacting production costs and market competitiveness. Traditional estimation methods, particularly Monte Carlo simulation, while reliable, become computationally prohibitive for complex modern circuits. Contemporary approaches, including importance sampling and machine learning techniques, face fundamental limitations in consistency across circuit topologies and practical validation. This work introduces YieldAgent, a novel Large Language Model (LLM)-powered framework that revolutionizes yield estimation through dynamic integration of multiple analytical strategies. YieldAgent employs a three-layer agent architecture to analyze circuit characteristics and historical data, optimizing estimation methods while balancing computational efficiency and precision. The framework incorporates Retrieval-Augmented Generation for domain knowledge integration and Tree-structured Parzen Estimators for dynamic hyperparameter optimization. Experimental validation across 12nm and 40nm technology nodes demonstrates that YieldAgent reduces computational overhead by up to 2.9× while maintaining or exceeding state-of-the-art accuracy. The system's ability to adapt across different circuit topologies and technology nodes establishes a new paradigm for scalable, intelligent yield estimation in electronic design automation.

Index Terms—Yield analysis, Multi-agent system, LLM

I. INTRODUCTION

Yield estimation serves as a cornerstone of semiconductor manufacturing, fundamentally impacting production optimization, cost management, and market competitiveness. The increasing complexity of modern integrated circuits, particularly in designs requiring high fault tolerance such as SRAM, has created unprecedented challenges in yield prediction. As process nodes continue to shrink and parameter spaces expand, traditional yield estimation methods face significant limitations in both accuracy and computational efficiency. The semiconductor industry's tight margins and competitive pressures make these limitations particularly acute, as even small improvements in yield estimation can translate to substantial financial benefits.

Traditional yield estimation methods, such as Monte Carlo (MC) simulation and Importance Sampling (IS), face growing challenges in modern semiconductor manufacturing. These methods struggle to balance efficiency and accuracy, especially with expanding parameter spaces and multiple failure regions [1]. As circuit complexity increases, their computational costs become prohibitive, demanding new solutions that reconcile accuracy with resource constraints.

[§] Denotes Corresponding Authors.

This work was supported by Key R&D Program of Shandong Province, China2022CXGC020208, Beijing Municipal Natural Science Foundation (Z230004), the Beijing Natural Science Foundation (L223004), Natural Science Foundation of China (62274008) and Beijing MSTC Program (Z231100007423019).

Recent AI advancements have introduced data-driven surrogate models for yield estimation. Techniques like Bayesian Yield Analysis (BYA) and Absolute Shrinkage Deep Kernel Learning (ASDK) show promise in modeling performance functions and predicting yield [2], [3]. However, they face challenges such as extensive training data requirements, optimization instabilities, and limited adaptability across circuit topologies. While IS-based methods have improved through mixture importance sampling and adaptive algorithms [4], [5], they remain computationally intensive and lack dynamic adaptability.

The rise of Large Language Models (LLMs) offers a promising solution, leveraging their reasoning capabilities and few-shot learning [6]. LLMs' ability to generalize from limited data is particularly valuable in yield estimation, where extensive data collection is impractical. Their adaptability across circuit topologies aligns with industry needs for scalable solutions. Recent LLM applications in electronic design automation, from HDL code generation [7]–[9] to design optimization [10] and debugging [11], [12], show potential, though their use in yield estimation remains underexplored.

This work introduces YieldAgent, an innovative LLM-powered framework that transforms yield estimation through intelligent strategy selection and resource allocation. YieldAgent implements a sophisticated three-layer agent architecture that leverages LLMs' reasoning capabilities for dynamic method integration and optimization. The framework offers three fundamental capabilities that distinguish it from existing approaches: dynamic budget-aware resource allocation for optimizing computational efficiency, precision-guided estimation that adapts strategies based on accuracy requirements, and topology-aware method selection that customizes approaches for specific circuit characteristics. The primary contributions of this work encompass three key innovations:

- **LLM-Powered Multi-Agent Architecture:** YieldAgent introduces a novel three-layer architecture (Strategy, Coordination, and Execution) that leverages LLMs for intelligent yield estimation. Through the integration of Retrieval-Augmented Generation (RAG) for domain knowledge and Chain-of-Thought (CoT) reasoning for strategy selection, the system achieves dynamic adaptation across diverse circuit topologies. This architecture enables specialized agents to collaborate effectively, from high-level planning to tactical execution.
- **Dynamic Knowledge Integration and Optimization:** We present a comprehensive framework combining historical performance data with live analysis. The system employs Tree-structured Parzen Estimators (TPE) for hyperparameter tuning and implements a sliding

window convergence mechanism for precision control. This integration enables automated strategy selection and resource allocation based on circuit characteristics and performance requirements.

- **Comprehensive Cross-Technology Validation:** Through extensive experimentation across 12nm and 40nm nodes, we demonstrate YieldAgent’s superior adaptability and efficiency. The framework achieves a 2.9× reduction in computational overhead while maintaining accuracy, validated through three key capabilities: budget-aware resource optimization, precision-guided estimation, and topology-aware method selection. This validation establishes YieldAgent’s effectiveness across different technology nodes and circuit complexities.

II. BACKGROUND

A. Problem Definition

The failure probability P_{fail} of a circuit is the likelihood that a performance metric Y falls into the failure region S :

$$P_{\text{fail}} = P(Y \in S) = \int I(x)f(x) dx \quad (1)$$

where $f(x)$ is the multivariate probability density function of the circuit’s parameters x , and $I(x)$ is an indicator function:

$$I(x) = \begin{cases} 1, & \text{if } x \in S, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

This integral is generally intractable because $I(x)$ is not analytically available, making direct evaluation of P_{fail} computationally infeasible.

B. Sampling-Based Yield Analysis

Monte Carlo Simulation (MC). Monte Carlo simulation estimates P_{fail} by randomly sampling N points $\{X_i\}_{i=1}^N$ from $f(x)$. The estimated failure probability is:

$$\hat{P}_{\text{fail}} = \frac{1}{N} \sum_{i=1}^N I(X_i) \quad (3)$$

As N increases, $\hat{P}_{\text{fail}}$ converges to P_{fail} . However, MC is inefficient for rare events, requiring a large number of samples when P_{fail} is very small [1].

Importance Sampling (IS). IS improves efficiency by modifying the sampling distribution to oversample the failure region S . Using a biased distribution $g(x)$ instead of $f(x)$, the failure probability becomes:

$$P_{\text{fail}} = \int I(x) \frac{f(x)}{g(x)} g(x) dx \quad (4)$$

The IS estimator is $\hat{P}_{\text{IS}} = \frac{1}{N} \sum_{i=1}^N w(X_i)I(X_i)$, where $w(X_i) = f(X_i)/g(X_i)$ ensures correct weighting of samples from $g(x)$. The optimal $g_{\text{opt}}(x) = I(x)f(x)/P_{\text{fail}}$ is impractical since P_{fail} and $I(x)$ are unknown, so practical methods approximate $g_{\text{opt}}(x)$ by shifting $f(x)$ towards S or using optimization-based approaches [4], [5].

C. AI-Enhanced Yield Analysis

AI introduces data-driven surrogate models to approximate complex performance functions in yield estimation. Examples include LRTA [13], which uses PCE; BYA [2], employing GP with entropy reduction in active learning; and ASDK [3], replacing GP with a deep kernel method and feature selection. Deep learning techniques like Normalizing Flows [14] model failure distributions.

Enhancements to IS methods focus on designing proposal distributions that closely approximate the failure distribution. MNIS [15] centers a normal distribution on the failure sample closest to the origin. GIS [16] introduces an Optimal Mean Shift Vector (OMSV) via gradient descent; FSIS [17] uses sensitivity analysis for efficiency; AIS [18] iteratively updates OMSV with new data. To address multi-region failures, HSCS [5] and ACS [19] employ cluster-based models and iteratively update the proposal distribution.

Despite these advancements, significant challenges remain. Traditional and AI-based methods often lack flexibility across diverse circuit topologies and face high computational costs due to extensive simulations or large training datasets. They typically use fixed strategies incapable of dynamically integrating multiple estimation methods or balancing computational load with precision requirements, limiting practical adoption [1], [5]. These gaps necessitate a new paradigm in yield estimation.

D. LLMs in Electronic Design Automation

LLMs have demonstrated promising capabilities across various EDA tasks. In hardware design, LLMs have enabled automated HDL code generation [7]–[9], design optimization [10], and debugging assistance [11], [12]. These applications leverage various LLM capabilities, including fine-tuning, prompt engineering, and domain adaptation techniques.

A key advantage of LLMs is their implicit Bayesian inference via in-context learning (ICL), allowing them to leverage prior knowledge and generalize from limited examples [6], [20]–[23]. This is particularly beneficial for yield estimation, where data scarcity due to large parameter spaces and high simulation costs poses significant challenges. Additionally, LLMs’ adaptability meets industrial needs for diverse circuit topologies and technology nodes.

However, applying LLMs to yield estimation faces unique challenges. Existing LLM frameworks lack specialized training data and struggle to integrate domain-specific knowledge, potentially leading to unreliable results in complex tasks. While LLMs excel at code generation, they often fail to produce functional methods for specialized applications. These limitations have historically impeded their effective use in yield estimation, especially where adaptability and computational efficiency are critical.

III. METHODOLOGY

A. System Overview and Design Principles

The fundamental challenge in yield estimation lies in balancing computational efficiency with estimation accuracy

Yield Analysis Workflow with YieldAgent

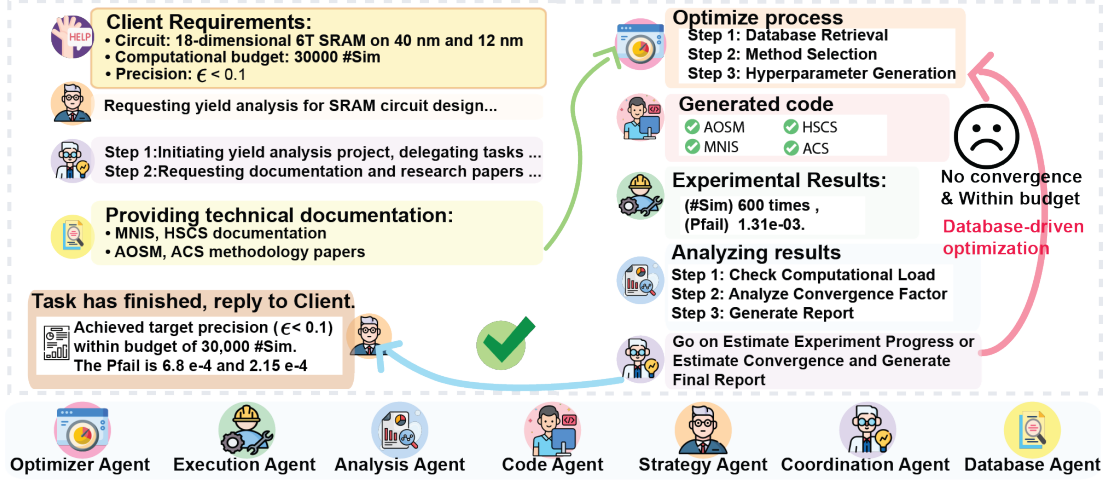


Fig. 1. Yield Analysis Workflow of YieldAgent

across diverse circuit topologies and technology nodes. YieldAgent addresses this challenge through three key design principles: adaptive resource allocation, precision-guided estimation, and topology-aware method selection. Fig. 1 illustrates the system architecture designed to implement these principles through a multi-agent framework.

The selection of a multi-agent architecture stems from the inherent complexity of yield estimation, where different stages of the process require distinct types of expertise and decision-making capabilities. Traditional approaches often treat yield estimation as a monolithic process, leading to inefficiencies in resource allocation and limited adaptability. By decomposing the process into specialized agents, YieldAgent enables dynamic adaptation to varying circuit characteristics while maintaining systematic coordination through LLM-based reasoning.

B. Multi-Agent Architecture

As illustrated in Fig. 2, YieldAgent implements a hierarchical three-layer architecture that separates strategic planning from tactical execution:

Strategic Layer: The Strategy Agent operates at the highest level, determining estimation goals and resource constraints. This agent leverages LLM reasoning capabilities to analyze circuit characteristics and historical performance data, developing high-level estimation strategies that align with specified precision requirements and computational budgets.

Coordination Layer: Two specialized agents operate at this level: the Coordination Agent manages task distribution and method selection, while the Database Agent handles knowledge integration and retrieval. This layer ensures coherent system-wide behavior by mediating between high-level strategies and concrete execution tasks.

Execution Layer: Four specialized agents handle specific tasks: (1) Code Generation Agent: Produces executable code for selected estimation methods, (2) Execution Agent: Manages simulation runs and resource utilization, (3) Analysis Agent: Evaluates results and computes convergence metrics, (4) Optimizer Agent: Tunes hyperparameters and adapts strategies in real-time.

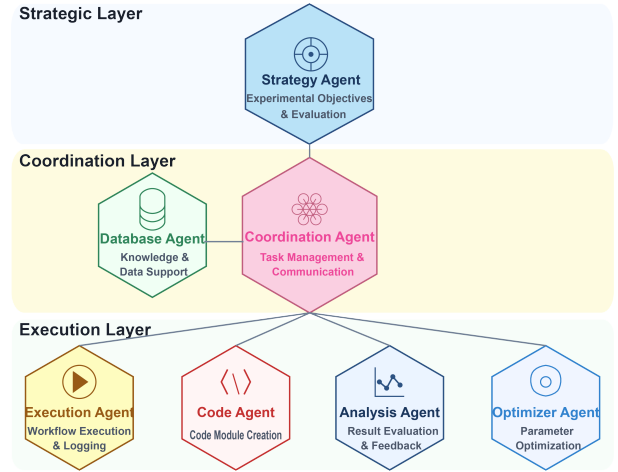


Fig. 2. YieldAgent Three-Layer Architecture Illustration.

C. Knowledge Integration and Method Selection

YieldAgent leverages a comprehensive knowledge base using Retrieval-Augmented Generation (RAG) to enhance its yield estimation capabilities. The knowledge base incorporates validated methodologies including MNIS [15], HSCS [5], AOSM [1], and ACS [19]. This integration addresses a critical limitation of LLMs: while they possess broad knowledge, they often lack specialized domain expertise required for technical tasks like yield estimation.

The Database Agent maintains performance profiles across different technology nodes and circuit topologies, enabling data-driven decision making. Through its implicit Bayesian inference capabilities [6], YieldAgent analyzes this historical data to adaptively select optimal strategies, functioning similarly to posterior updates in Bayesian methods. Validated across multiple technology nodes (e.g., Fig. 3), the database provides performance profiles that support informed, data-driven decisions.

The Coordination Agent combines database insights with real-time performance metrics to dynamically adjust estima-

tion strategies. This adaptive framework ensures robust and reliable yield estimation by: (1) Analyzing circuit characteristics to identify suitable estimation methods (2) Monitoring convergence rates to adjust computational resource allocation (3) Adapting strategy combinations based on real-time performance feedback.

D. Dynamic Hyperparameter Optimization

Empirical analysis reveals that direct transfer of hyperparameters across technology nodes often leads to suboptimal performance. As shown in Fig. 4, methods optimized for 40nm technology exhibit significant performance degradation when applied to 12nm circuits without adjustment. The failure probability (P_{fail}) curves demonstrate substantial gaps between 40nm-optimized configurations and reference results on 12nm nodes.

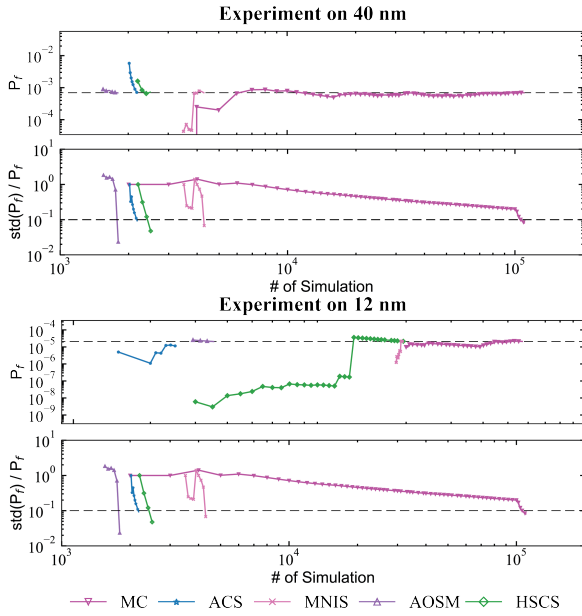


Fig. 3. Performance Comparison of Yield Estimation Methods Across 40nm and 12nm Nodes

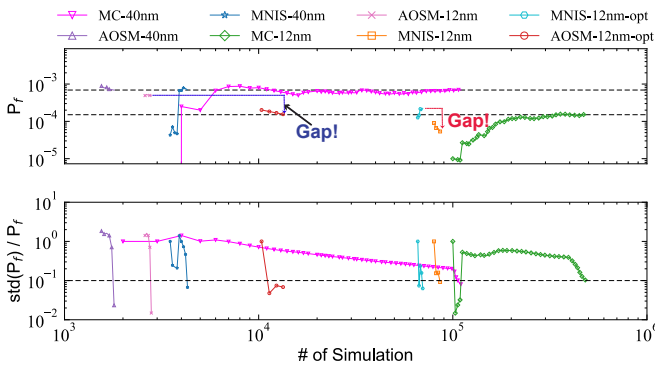


Fig. 4. Yield Analysis on 40nm and 12nm Nodes for MNIS and AOSM

YieldAgent addresses this challenge through a Tree-structured Parzen Estimator (TPE) that dynamically optimizes hyperparameters based on real-time performance metrics. The TPE algorithm models the relationship between hyperparameters

and estimation performance using two probability density functions:

$$l(x) = p(x|y \leq y^*) \text{ and } g(x) = p(x|y > y^*) \quad (5)$$

where y^* denotes the performance threshold. New hyperparameters are selected by maximizing the expected improvement ratio:

$$x_{t+1} = \arg \max_x (l(x)/g(x)) \quad (6)$$

E. Statistical Convergence Analysis

YieldAgent employs a robust convergence analysis framework designed for dynamic, multi-model scenarios. The framework uses a sliding window statistical approach to assess the stability of the entire estimation process. For a window of size k , we calculate **Relative Variation**:

$$RV_k = \frac{\sigma_k}{\bar{x}} \quad (7)$$

where σ_k is the standard deviation and $\bar{x}$ is the mean of results within the window. Convergence is achieved when $RV_k < \epsilon$, where ϵ is the predefined precision threshold. This approach offers several advantages such as, (1) Robustness to local fluctuations, (2) Adaptive sensitivity through relative variation, (3) Computational efficiency via incremental updates. The sliding window approach provides a reliable convergence assessment than traditional metrics like Figure of Merit (FoM), particularly in multiple models and iterative refinement scenarios.

F. Integrated Workflow

YieldAgent leverages its modular multi-agent architecture to address yield estimation challenges by dynamically selecting methods, optimizing hyperparameters, and efficiently allocating computational resources. This is exemplified in the estimation of yield for a 6T-SRAM circuit at both 40nm and 12nm technology nodes, constrained by a computational budget of 30,000 simulations and a target precision of $\epsilon < 0.1$, as shown in Fig. 1.

The workflow begins with the user specifying circuit design parameters and requirements. The **Strategy Agent** analyzes this input to develop an initial estimation plan. This analysis encompasses circuit topology and complexity, technology node characteristics, precision requirements, and computational constraints. These factors form the basis for subsequent optimization decisions.

The **Database Agent** then retrieves relevant historical performance data for various estimation methods (MNIS, HSCS, AOSM, and ACS) across similar scenarios. Based on this information, the **Coordination Agent** selects optimal method combinations for each technology node through a systematic process. This process involves initial method selection based on historical performance, followed by resource allocation planning across selected methods, and concludes with the definition of intermediate evaluation points for strategy adjustment.

During the simulation phase, the **Code Generation Agent** produces executable code tailored to the selected methods.

This code incorporates method-specific implementations, initialization parameters, and convergence checks. The **Execution Agent** then manages simulation runs using initial hyperparameters optimized by the **Optimizer Agent**. The execution process follows a formal optimization structure:

- 1) For each method M_i in selected set:
 - a) Initialize: $\theta_i = \text{TPE}_{\text{init}}(M_i, \text{historical_data})$
 - b) Execute: $\text{results}_i = M_i(\theta_i, \text{circuit_params})$
 - c) Update: $\theta_i = \text{TPE}_{\text{update}}(\text{results}_i, \theta_i)$

The **Analysis Agent** periodically evaluates intermediate results using the sliding window statistical approach to calculate relative variation (RV_k) and assess convergence:

$$RV_k = \frac{\sigma_k}{\bar{x}} < \epsilon \quad (8)$$

When convergence criteria are not met, the system initiates an iterative refinement process. The Optimizer Agent adjusts hyperparameters using TPE, while the Coordination Agent modifies method weights or combinations. Simultaneously, the Strategy Agent may reallocate computational resources to optimize performance.

The optimization process employs a dynamic feedback mechanism where hyperparameter configurations are adjusted to maximize expected improvements in precision:

$$\theta_{t+1} = \arg \max_{\theta} \mathbb{E}[\text{Improvement}(\theta | \mathcal{D}_t)] \quad (9)$$

where $\mathcal{D}_t$ represents the historical performance data up to iteration t . Methods exhibiting suboptimal performance undergo systematic enhancement through additional hyperparameter optimization, replacement with alternative methods from the database, or integration with complementary approaches. This adaptive strategy ensures continuous improvement of estimation accuracy while maintaining computational efficiency.

The integrated workflow demonstrates YieldAgent's capability to dynamically optimize estimation strategies while balancing computational efficiency with precision requirements. The system adapts seamlessly to diverse process nodes and circuit characteristics through systematic coordination of its multi-agent components. This comprehensive approach establishes a robust framework for yield estimation across varying technological contexts and precision demands.

The effectiveness of this workflow is validated through comprehensive experiments across different technology nodes and precision requirements, as detailed in Section IV. These experiments demonstrate quantitative improvements in both computational efficiency and estimation accuracy, establishing YieldAgent's practical utility in modern semiconductor manufacturing contexts.

IV. EXPERIMENTS

The experimental validation of YieldAgent focuses on three fundamental capabilities: computational resource optimization, precision-guided estimation, and technology node adaptation. A 6T-SRAM circuit serves as the primary benchmark, chosen for its sensitivity to process variations and industrial relevance. The evaluation spans multiple technology nodes and precision requirements to demonstrate versatility and effectiveness.

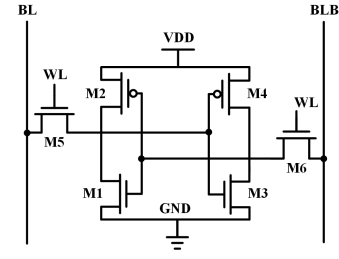


Fig. 5. Structure of the 6T-SRAM bit cell used in experiments.

A. Experimental Setup

YieldAgent's implementation utilizes Microsoft's AutoGen [24] multi-agent framework integrated with OpenAI's GPT-4o-mini model. The system operates on an AMD EPYC 7601 CPU platform with 62GB RAM, employing HSPICE for circuit simulation. All convergence evaluations utilize a sliding window size of $k = 5$ to ensure statistical stability.

The 6T-SRAM bit cell, illustrated in Fig. 5, serves as an ideal test vehicle for yield estimation due to its sensitivity to process variations and widespread deployment in semiconductor research. The experiments span TSMC 12nm and SMIC 40nm technology nodes to assess YieldAgent's adaptability across different manufacturing processes.

Process variations are modeled through three critical transistor parameters: oxide thickness (t_{ox}), threshold voltage (V_{th}), and carrier mobility (μ). These parameters significantly influence transistor performance metrics, including gate capacitance, switching characteristics, and drive current. SRAM read and write delay serve as key performance metrics for evaluating the impact of these variations.

B. Budget-Aware Capability Evaluation

The evaluation of YieldAgent's resource optimization capabilities employs three distinct computational budgets: Low (3,000 simulations), Medium (6,000 simulations), and High (10,000 simulations). The analysis compares YieldAgent against traditional Monte Carlo simulation and four state-of-the-art methods: MNIS [15], AOSM [1], ACS [19], and HSCS [5]. A reference failure rate of 6.91×10^{-4} is established through Monte Carlo simulation using 110,000 samples.

Table I presents comprehensive comparative results across budget levels. Under the Low budget constraint, YieldAgent achieves 4.1% relative error using 2,890 simulations through dynamic combination of MNIS, AOSM, and ACS methods. This performance significantly surpasses the best single-method approach (ACS: 8.6% error, 2,980 simulations).

In the Medium budget scenario, YieldAgent demonstrates superior method integration capabilities by combining MNIS with iterative AOSM refinements. This approach achieves 2.96% relative error using 5,550 simulations, while fixed methods exhibit errors ranging from 3.4% to 6.7%. The High budget level further showcases YieldAgent's strategic prowess, where its combination of MNIS and AOSM iterations yields 1.8% relative error with 9,200 simulations, compared to AOSM's 3.6% error using 9,950 simulations.

TABLE I
YIELD ANALYSIS WITH VARYING BUDGETS

Budget	Method	# Sim	Fail Rate	Rel. Err.
–	MC	110,000	6.91×10^{-4}	–
Low	MNIS	2,500	6.16×10^{-4}	10.8%
	AOSM	2,920	6.04×10^{-4}	12.5%
	ACS	2,980	6.32×10^{-4}	8.6%
	HSCS	2,050	5.81×10^{-4}	15.8%
	YieldAgent	2,890	6.62×10^{-4}	4.1%
Medium	MNIS	5,500	6.67×10^{-4}	3.4%
	AOSM	5,950	6.65×10^{-4}	3.7%
	ACS	4,320	6.59×10^{-4}	4.6%
	HSCS	5,900	7.37×10^{-4}	6.7%
	YieldAgent	5,550	6.70×10^{-4}	2.96%
High	MNIS	6,800	6.42×10^{-4}	7.1%
	AOSM	9,950	6.66×10^{-4}	3.6%
	ACS	9,680	6.58×10^{-4}	4.7%
	HSCS	9,750	7.18×10^{-4}	3.9%
	YieldAgent	9,200	6.79×10^{-4}	1.8%

Fail Rate: Estimated failure rate; Rel. Err.: Relative estimation error compared to MC method.

C. Precision-Aware Capability Evaluation

This phase of experimentation assesses YieldAgent’s ability to meet varying precision requirements defined by convergence factors $\epsilon = \{0.1, 0.05, 0.01\}$. The evaluation employs relative error and simulation count as primary metrics, with convergence determined when the relative variation in yield estimates falls below the specified ϵ threshold across the sliding window.

Table II quantifies YieldAgent’s performance across precision levels. At $\epsilon = 0.1$, the framework achieves 2.7% relative error using 5,550 simulations through ACS-MNIS method integration, while HSCS requires 16,100 simulations for comparable accuracy (2.6% error). Under stricter precision requirements ($\epsilon = 0.05, 0.01$), YieldAgent maintains superior efficiency through dynamic method selection, achieving relative errors of 1.0% and 0.4% with 8,760 and 15,600 simulations respectively. This performance represents a significant improvement over single-method approaches, which require 20,800 and 23,300 simulations for similar precision levels.

TABLE II
YIELD ANALYSIS WITH VARYING CONVERGENCE FACTORS

ϵ	Method	Fail Rate	# Sim	Rel. Err.
0.1	ACS	6.40×10^{-4}	5,580 (1.0 $\times$)	7.3%
	AOSM	6.61×10^{-4}	6,750 (1.22 $\times$)	4.3%
	HSCS	6.69×10^{-4}	16,100 (2.90 $\times$)	2.6%
	YieldAgent	6.79×10^{-4}	5,550 (1$\times$)	2.7%
0.05	ACS	6.53×10^{-4}	7,140 (0.82 $\times$)	5.5%
	AOSM	7.03×10^{-4}	9,800 (1.12 $\times$)	1.7%
	HSCS	6.74×10^{-4}	20,800 (2.37 $\times$)	2.5%
	YieldAgent	6.85×10^{-4}	8,760 (1$\times$)	1.0%
0.01	ACS	6.83×10^{-4}	13,120 (0.84 $\times$)	1.2%
	HSCS	7.01×10^{-4}	14,350 (0.92 $\times$)	1.4%
	MNIS	6.85×10^{-4}	23,300 (1.5 $\times$)	0.9%
	YieldAgent	6.88×10^{-4}	15,600 (1$\times$)	0.4%

D. Topology-Aware Capability Evaluation

The final experimental phase examines YieldAgent’s adaptation capabilities across TSMC 12nm and SMIC 40nm technology nodes, where process variations and failure mechanisms

differ significantly. The 12nm node exhibits tighter threshold voltage variation ($\sigma_{V_{th}} = 30\text{mV}$) and more complex failure regions compared to 40nm ($\sigma_{V_{th}} = 45\text{mV}$), presenting distinct estimation challenges.

Table III presents comparative results across nodes and precision requirements. For the 40nm node, YieldAgent achieves superior estimation accuracy through strategic method combinations: ACS-MNIS for $\epsilon = 0.1$ (2.7% error, 20x simulation reduction), and ACS-HSCS for stricter requirements (1.0% and 0.4% error, with 12.67x and 7.12x reductions).

The framework adapts differently for 12nm technology, employing ACS-AOSM combinations to handle complex failure distributions. This strategy achieves relative errors of 7.6%, 3.4%, and 1.9% while reducing simulation counts by factors of 12.34x, 9.03x, and 5.84x respectively. The performance differential between nodes demonstrates YieldAgent’s topology-aware capabilities, selecting different method combinations based on technology-specific characteristics: ACS-MNIS/HSCS for 40nm’s wider variation spread, and ACS-AOSM for 12nm’s complex failure regions.

TABLE III
YIELDAGENT’S ESTIMATION WITH TECHNOLOGY NODES AND CONVERGENCE FACTORS ϵ

Node	ϵ	Fail Rate	# Sim	Rel. Err.	Ensembled Method
40 nm	MC	6.91×10^{-4}	111,000	–	–
	0.1	6.79×10^{-4}	5,550(20 $\times$)	2.7%	ACS, MNIS
	0.05	6.85×10^{-4}	8,760(12.67 $\times$)	1.0%	ACS, HSCS
	0.01	6.88×10^{-4}	15,600(7.12 $\times$)	0.4%	ACS, HSCS
12 nm	MC	2.11×10^{-5}	569,000	–	–
	0.1	2.27×10^{-5}	46,000(12.34 $\times$)	7.6%	ACS, AOSM
	0.05	2.18×10^{-5}	63,000(9.03 $\times$)	3.4%	ACS, AOSM
	0.01	2.15×10^{-5}	87,400(5.84 $\times$)	1.9%	ACS, AOSM

V. CONCLUSION

This research addresses significant challenges in yield estimation for semiconductor manufacturing through the introduction of YieldAgent, the first LLM-powered multi-agent framework for dynamic yield estimation. Through its novel three-layer architecture integrating LLM reasoning capabilities with domain-specific knowledge, YieldAgent demonstrates superior adaptability and efficiency across diverse circuit topologies and technology nodes. Experimental validation across 12nm and 40nm nodes shows that YieldAgent reduces computational overhead by up to 2.9 $\times$ while maintaining or exceeding state-of-the-art accuracy, achieved through three key innovations: dynamic budget-aware resource allocation, precision-guided estimation, and topology-aware method selection. The framework’s success in balancing computational efficiency with estimation precision, while adapting to varying technological contexts, establishes a foundation for future innovations in semiconductor manufacturing, with potential extensions to other areas of electronic design automation and integration with emerging AI techniques for further enhancement of yield estimation capabilities.

REFERENCES

- [1] J. Yao, Z. Ye, and Y. Wang, "An efficient sram yield analysis and optimization method with adaptive online surrogate modeling," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 23, no. 7, pp. 1245–1253, 2014.
- [2] M. Wang, F. Yang, C. Yan, X. Zeng, and X. Hu, "Efficient bayesian yield optimization approach for analog and sram circuits," in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*. IEEE, 2017, pp. 1–6.
- [3] S. Yin, G. Dai, and W. W. Xing, "High-dimensional yield estimation using shrinkage deep features and maximization of integral entropy reduction," in *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, 2023, pp. 283–289.
- [4] L. Dolecek, M. Qazi, D. Shah, and A. Chandrakasan, "Breaking the simulation barrier: Sram evaluation through norm minimization," in *2008 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 2008, pp. 322–329.
- [5] W. Wu, S. Bodapati, and L. He, "Hyperspherical clustering and sampling for rare event analysis with multiple failure region coverage," in *Proceedings of the 2016 on International Symposium on Physical Design*, 2016, pp. 153–160.
- [6] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, "An explanation of in-context learning as implicit bayesian inference," *arXiv preprint arXiv:2111.02080*, 2021.
- [7] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 2023, pp. 1–6.
- [8] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, "Chipgpt: How far are we from natural language hardware design," *arXiv preprint arXiv:2305.14019*, 2023.
- [9] H. Pearce, B. Tan, and R. Karri, "Dave: Deriving automatically verilog from english," in *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*, 2020, pp. 27–32.
- [10] Z. Pei, H.-L. Zhen, M. Yuan, Y. Huang, and B. Yu, "Betterv: Controlled verilog generation with discriminative guidance," *arXiv preprint arXiv:2402.03375*, 2024.
- [11] Y. Tsai, M. Liu, and H. Ren, "Rtlfixer: Automatically fixing rtl syntax errors with large language model," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [12] X. Yao, H. Li, T. H. Chan, W. Xiao, M. Yuan, Y. Huang, L. Chen, and B. Yu, "Hdldebugger: Streamlining hdl debugging with large language models," *arXiv preprint arXiv:2403.11671*, 2024.
- [13] X. Shi, H. Yan, Q. Huang, J. Zhang, L. Shi, and L. He, "Meta-model based high-dimensional yield analysis using low-rank tensor approximation," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.
- [14] Y. Liu, G. Dai, and W. W. Xing, "Seeking the yield barrier: High-dimensional sram evaluation through optimal manifold," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [15] L. Dolecek, M. Qazi, D. Shah, and A. Chandrakasan, "Breaking the simulation barrier: Sram evaluation through norm minimization," in *2008 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 2008, pp. 322–329.
- [16] T. Haine, J. Segers, D. Flandre, and D. Bol, "Gradient importance sampling: An efficient statistical extraction methodology of high-sigma sram dynamic characteristics," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 195–200.
- [17] W. Hu, Z. Wang, S. Yin, Z. Ye, and Y. Wang, "Sensitivity importance sampling yield analysis and optimization for high sigma failure rate estimation," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 895–900.
- [18] X. Shi, F. Liu, J. Yang, and L. He, "A fast and robust failure analysis of memory circuits using adaptive importance sampling method," in *Proceedings of the 55th Annual Design Automation Conference*, 2018, pp. 1–6.
- [19] X. Shi, H. Yan, J. Wang, X. Xu, F. Liu, L. Shi, and L. He, "Adaptive clustering and sampling for high-dimensional and multi-failure-region sram yield analysis," in *Proceedings of the 2019 International Symposium on Physical Design*, 2019, pp. 139–146.
- [20] S. Mirchandani, F. Xia, P. Florence, B. Ichter, D. Driess, M. G. Arenas, K. Rao, D. Sadigh, and A. Zeng, "Large language models as general pattern machines," *arXiv preprint arXiv:2307.04721*, 2023.
- [21] T. Liu, N. Astorga, N. Seedat, and M. van der Schaar, "Large language models to enhance bayesian optimization," *arXiv preprint arXiv:2402.03921*, 2024.
- [22] S. Yang, H. Zhao, S. Zhu, G. Zhou, H. Xu, Y. Jia, and H. Zan, "Zhongjing: Enhancing the chinese medical capabilities of large language model through expert feedback and real-world multi-turn dialogue," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 368–19 376.
- [23] G. Chen, K. Zhu, S. Kim, H. Zhu, Y. Lai, B. Yu, and D. Z. Pan, "Llm-enhanced bayesian optimization for efficient analog layout constraint generation," *arXiv preprint arXiv:2406.05250*, 2024.
- [24] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversation."